Chapter5:Database Architecture

Database System Concepts

University of Guilan

Yasaman Boreshban

Fall 2025

Motivation for Database Architecture

- Need for a unified data-oriented architecture
- Data should be viewable independently from implementation complexity
- Lack of consensus on database architecture in early years
- ANSI/SPARC three-level architecture proposal
- Three levels: data definition & control, logical level, physical storage level

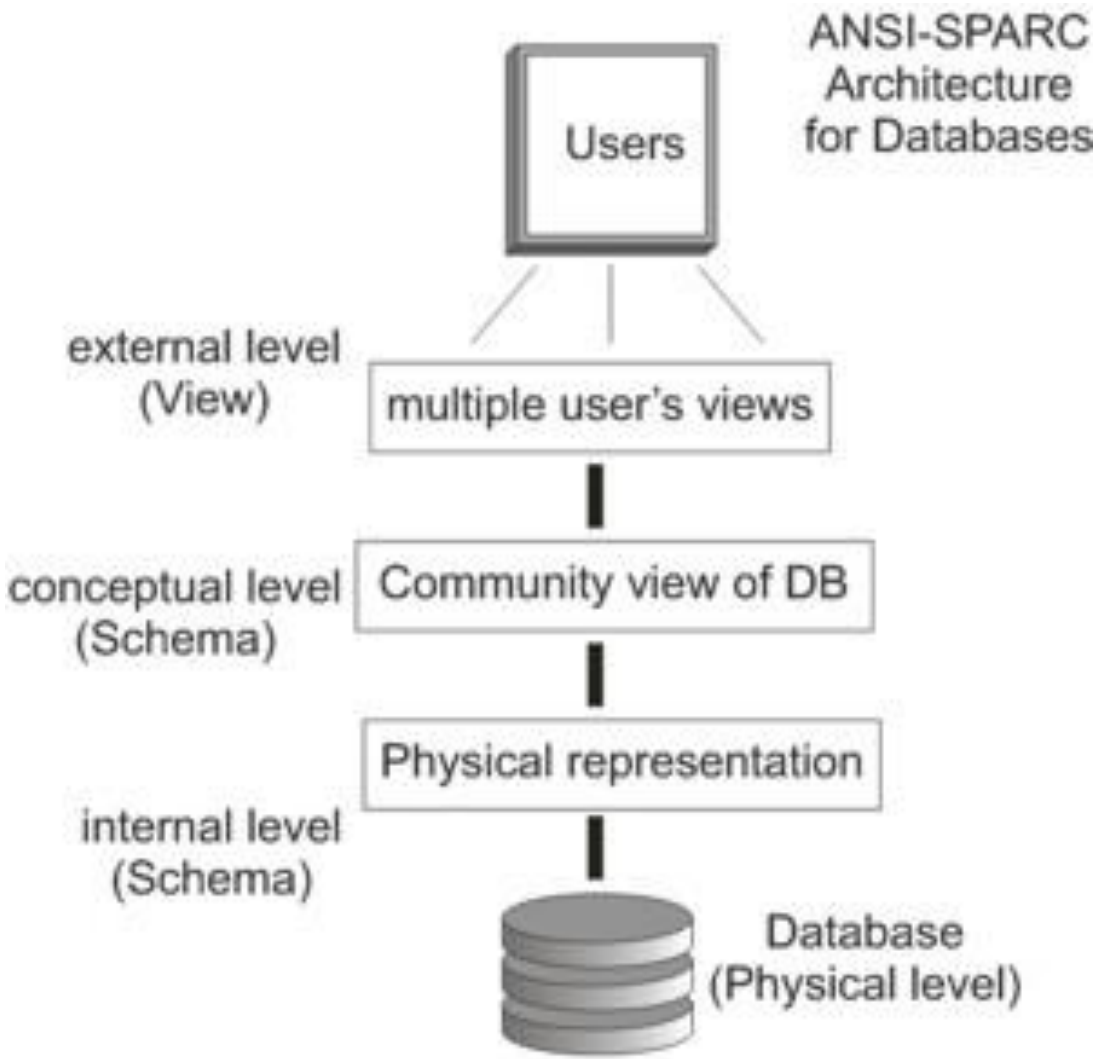
ANSI/SPARC Three-Level Architecture

- **External Level:** multiple user views with high-level languages
- **Conceptual Level:** unified logical view of the entire database
- **Internal Level:** physical storage and file organization

- Data independence between levels

- DBMS maps and transforms data across all three levels

ANSI


ANSI-SPARC
Architecture
for Databases
Users
external level
(View)
multiple user's views
conceptual level
(Schema)
Community view of DB
Physical representation
internal level
(Schema)
Database
(Physical level)

Conceptual View

- The designer’s view of data to be stored (not necessarily finally stored)
- A comprehensive view covering all users’ requirements
- Defined in the conceptual (perceptual) environment based on a specific data structure
- Designed using fundamental structural elements
- After design, the Conceptual Schema is described
- A program containing DDL and DCL commands, not DML
- The conceptual schema is given to the DBMS and stored in the system catalog

Conceptual View (Continued)

- CREATE TABLE S1 …
- CREATE TABLE S2 …
- CREATE TABLE S3 …

- In the conceptual view, S1, S2, and S3 are base tables
- The conceptual schema is the definition of these tables
- These tables are called base tables

Conceptual View (Continued)

- SYSTABLES table as an example
- User (implementer) issues CREATE TABLE STT ...
- System inserts a record into SYSTABLES
- User (implementer) issues DROP TABLE STCOT ...
- System deletes the corresponding record from SYSTABLES

Developer : **CREATE TABLE** STT ...

System : **INSERT INTO** SYSTABLES
VALUES ('STT' , 'c1' , 'd1' , 5 , 'STID' , ...)

Developer : **DROP TABLE** STCOT ...

System : **DELETE FROM** SYSTABLES
WHERE TNAME = 'STCOT'

Internal View

- The internal view represents the DBMS view (or the designer's view in physical design)
- It concerns the data stored physically
- Defined at the logical (or sometimes virtual) filing level
- Based on one or several file structures
- 1:1 default mapping (one table : one file)
- 1:N mapping (multiple tables : one file)
- N:1 mapping (one table : multiple files)
- Defines the logical files of the database

External View

- The external view represents the user's specific view of stored data
- A partial view that includes the data needs of a particular user
- Defined at the abstraction level based on a specific data structure
- External views are defined based on the conceptual view
- One user may have multiple external views
- Multiple users may share one external view

External View (Continued)

- Description of the external view forms the External Schema
- The external user writes a “program” that contains data-definition commands and a limited number of data-control commands
- The external schema is stored in the catalog
- In table-based systems, the external view behaves like a table but is virtual and not stored
- The external view is a window through which the user sees only their allowed portion of the data

Mappings Between Levels in External Schema Operations

- Mappings (or transformations) between levels during DB operations:

- E/C : External to Conceptual Mapping
- C/I : Conceptual to Internal Mapping

- External View
  - E/C
- Conceptual View
  - C/I
- Internal View

Operations in the External Schema

- Retrieval: The user is allowed to retrieve data within the limits of their own view.

- Storage operations (authorized by Admin):
  - Insert
  - Delete
  - Update

- Each command executed on the external schema (external view)
- is transformed into corresponding command(s) on the conceptual schema,
- then into the internal schema operations,
- and finally into physical file operations.

Data Independence

- Concept of Data Independence (DI):
- Programs at the external level remain unaffected by changes in the lower levels of the DB architecture.

- Why shouldn't programs change?
- Because modifying programs increases the cost of development, maintenance, and redevelopment.

- Types of Data Independence:
  - Physical Data Independence (PDI)
  - Logical Data Independence (LDI)

Physical Data Independence (PDI)

- Physical Data Independence means that user programs remain unaffected by changes in the internal schema of the database.

- Internal schema changes include file-level modifications such as:
  - file structure, record length, storage organization on disk, and changes handled either physically or by the DBMS.

- Physical Data Independence is fully supported in databases because users interact with views, which exist at a level far above file-level concerns.

Logical Data Independence (LDI)

- Logical Data Independence means user programs remain unaffected by changes in the conceptual schema of the database.
- Most DBMSs provide this independence to a large extent, though not fully.
- Changes in the conceptual schema may include:
  - Database growth (DB Growth)
  - Database restructuring (DB Restructuring)

End of Chapter